

Detailed Low-Level Design (DLD)

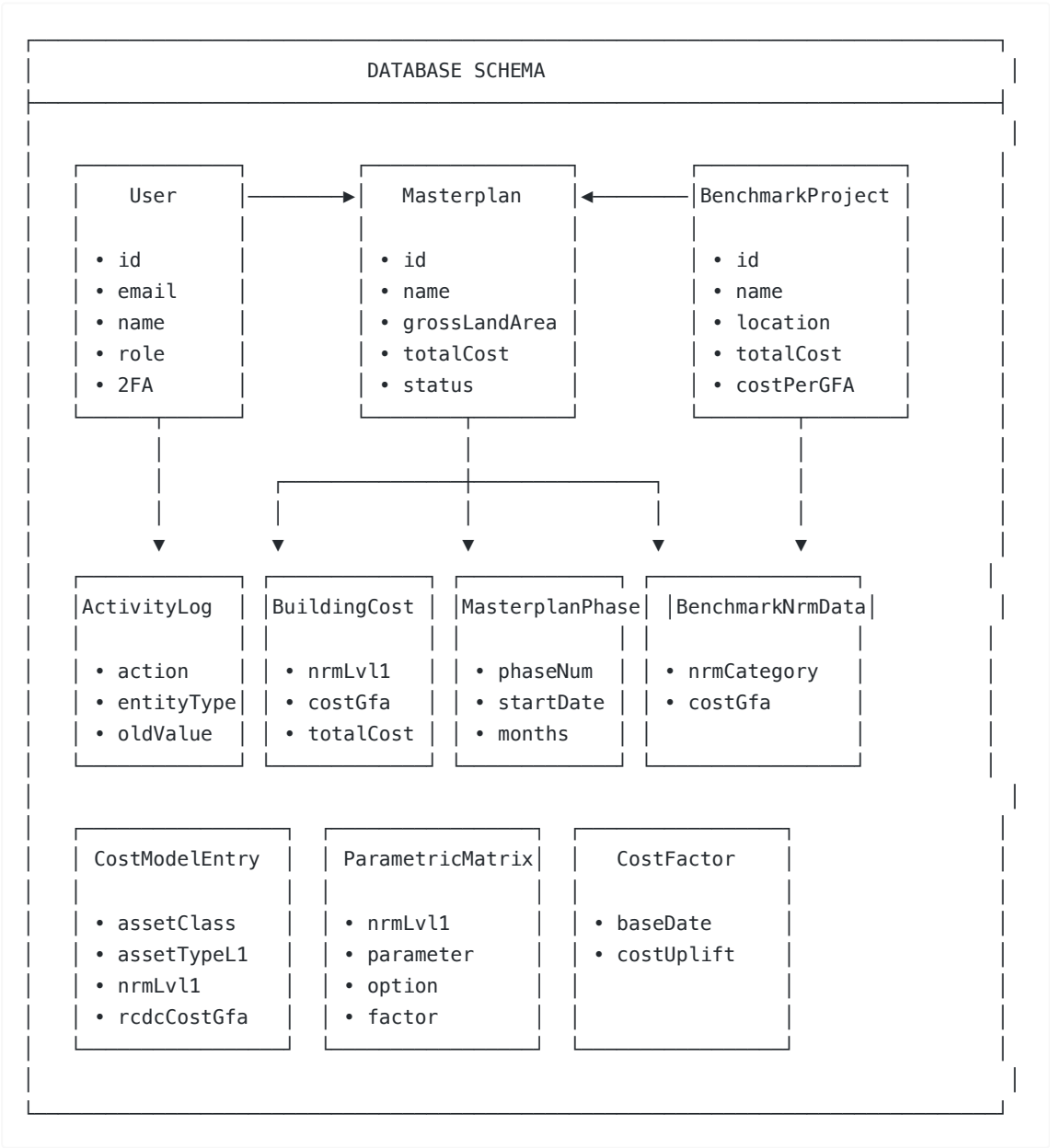
ROSHN Parametric Masterplan Modelling Platform

Version: 1.0 **Date:** 5 January 2026 **Status:** Production Ready

Note: The application stack is certified production-ready. Any future enhancements or modifications based on ROSHN feedback will be implemented as iterative improvements and do not affect the current production readiness status.

1. Database Schema

1.1 Entity Relationship Diagram



1.2 Core Tables

Users Table

```
CREATE TABLE users (  
  id                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  email             VARCHAR(255) UNIQUE NOT NULL,  
  name              VARCHAR(255),  
  role              user_role DEFAULT 'VIEWER',  
  password_hash     VARCHAR(255),  
  password_changed_at  TIMESTAMP,  
  must_change_password  BOOLEAN DEFAULT false,  
  failed_login_attempts INTEGER DEFAULT 0,  
  locked_until       TIMESTAMP,  
  two_factor_enabled  BOOLEAN DEFAULT false,  
  two_factor_secret   VARCHAR(255),  
  backup_codes       TEXT[],  
  allowed_countries  TEXT[],  
  allowed_developers TEXT[],  
  department         VARCHAR(255),  
  is_active          BOOLEAN DEFAULT true,  
  last_login_at      TIMESTAMP,  
  created_at         TIMESTAMP DEFAULT NOW(),  
  updated_at         TIMESTAMP DEFAULT NOW()  
);  
  
CREATE INDEX idx_users_email ON users(email);  
CREATE INDEX idx_users_role  ON users(role);  
CREATE INDEX idx_users_is_active ON users(is_active);
```

Masterplans Table

```
CREATE TABLE masterplans (  
  id                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  name              VARCHAR(255) NOT NULL,  
  description        TEXT,  
  gross_land_area    DECIMAL(12,2) NOT NULL,  
  calculated_plot_area DECIMAL(12,2) NOT NULL,  
  balance_external_area DECIMAL(12,2) NOT NULL,  
  total_units        INTEGER NOT NULL,  
  parking_spaces     INTEGER NOT NULL,  
  contingency         DECIMAL(15,2) NOT NULL,  
  total_cost         DECIMAL(15,2) NOT NULL,  
  cost_per_gfa       DECIMAL(12,2) NOT NULL,  
  asset_class        VARCHAR(255) NOT NULL,  
  asset_type_l1      VARCHAR(255) NOT NULL,  
  asset_form_l2      VARCHAR(255),  
  status             masterplan_status DEFAULT 'DRAFT',  
  version            INTEGER DEFAULT 1,  
  created_by_id      UUID REFERENCES users(id),
```

```

    number_of_phases      INTEGER DEFAULT 1,
    benchmark_project_id  UUID REFERENCES benchmark_projects(id),
    country               VARCHAR(255),
    developer             VARCHAR(255),
    is_public             BOOLEAN DEFAULT false,
    created_at            TIMESTAMP DEFAULT NOW(),
    updated_at            TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_masterplans_created_by ON masterplans(created_by_id);
CREATE INDEX idx_masterplans_status ON masterplans(status);
CREATE INDEX idx_masterplans_asset_class ON masterplans(asset_class);
CREATE INDEX idx_masterplans_country ON masterplans(country);
CREATE INDEX idx_masterplans_benchmark ON masterplans(benchmark_project_id);

```

Cost Model Entries Table

```

CREATE TABLE cost_model_entries (
    id                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    asset_class       VARCHAR(255) NOT NULL,
    asset_type_l1     VARCHAR(255) NOT NULL,
    asset_form_l2     VARCHAR(255),
    price_point       VARCHAR(255),
    nrm_lvl1          VARCHAR(255) NOT NULL,
    nrm_lvl2          VARCHAR(255),
    nrm_lvl3          VARCHAR(255),
    unit_of_measurement VARCHAR(50),
    sar_per_uom       DECIMAL(12,2),
    rcdc_cost_gfa     DECIMAL(12,2) NOT NULL,
    benchmarked_cost_gfa DECIMAL(12,2),
    created_at        TIMESTAMP DEFAULT NOW(),
    updated_at        TIMESTAMP DEFAULT NOW(),

    UNIQUE(asset_class, asset_type_l1, asset_form_l2, price_point, nrm_lvl1)
);

CREATE INDEX idx_cost_model_asset_class ON cost_model_entries(asset_class);
CREATE INDEX idx_cost_model_asset_type ON cost_model_entries(asset_type_l1);
CREATE INDEX idx_cost_model_nrm ON cost_model_entries(nrm_lvl1);

```

1.3 Enum Types

```

CREATE TYPE user_role AS ENUM ('ADMIN', 'DEVELOPMENT_MANAGER', 'VIEWER');
CREATE TYPE team_role AS ENUM ('MANAGER', 'VIEWER');
CREATE TYPE masterplan_status AS ENUM ('DRAFT', 'ACTIVE', 'ARCHIVED', 'APPROVED');
CREATE TYPE file_category AS ENUM ('COST_MODEL_CSV', 'BENCHMARK_CSV', 'DOCUMENT', 'IMAGE');

```

2. API Specification

2.1 REST API Endpoints

Authentication APIs

Method	Endpoint	Description
POST	/api/auth/signin	User login
POST	/api/auth/signout	User logout
GET	/api/auth/session	Get current session
POST	/api/auth/2fa/setup	Setup 2FA
POST	/api/auth/2fa/verify	Verify 2FA code
POST	/api/auth/2fa/disable	Disable 2FA
GET	/api/auth/sso-status	Check SSO configuration

User APIs

Method	Endpoint	Description
GET	/api/user/me	Get current user profile
GET	/api/admin/users	List all users (admin)
POST	/api/admin/users	Create new user (admin)
PUT	/api/admin/users/{id}	Update user (admin)
DELETE	/api/admin/users/{id}	Delete user (admin)

Masterplan APIs

Method	Endpoint	Description
GET	/api/masterplans	List masterplans
POST	/api/masterplans	Create masterplan
GET	/api/masterplans/{id}	Get masterplan details
PUT	/api/masterplans/{id}	Update masterplan
DELETE	/api/masterplans/{id}	Delete masterplan

Benchmark Project APIs

Method	Endpoint	Description
GET	/api/benchmark-projects	List benchmark projects
POST	/api/benchmark-projects	Create benchmark project

GET	/api/benchmark-projects/[id]	Get project details
PUT	/api/benchmark-projects/[id]	Update project
DELETE	/api/benchmark-projects/[id]	Delete project
GET	/api/benchmark-filter-options	Get filter options

Cost Model APIs

Method	Endpoint	Description
GET	/api/cost-model-entries	List cost model entries
POST	/api/cost-model-entries	Create/Import entries

Admin APIs

Method	Endpoint	Description
GET	/api/admin/audit-logs	Get audit logs
GET	/api/admin/settings	Get system settings
PUT	/api/admin/settings	Update system settings

2.2 API Response Format

Success Response:

```
{
  "success": true,
  "data": { ... },
  "message": "Operation completed successfully"
}
```

Error Response:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid input data",
    "details": [ ... ]
  }
}
```

2.3 Server Actions

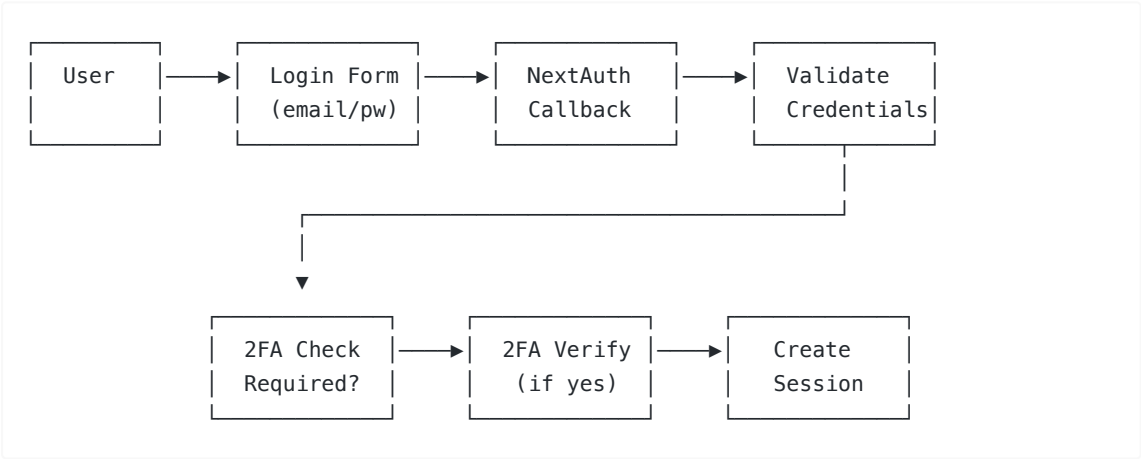
Server Actions handle business logic directly from components:

Action	File	Description
--------	------	-------------

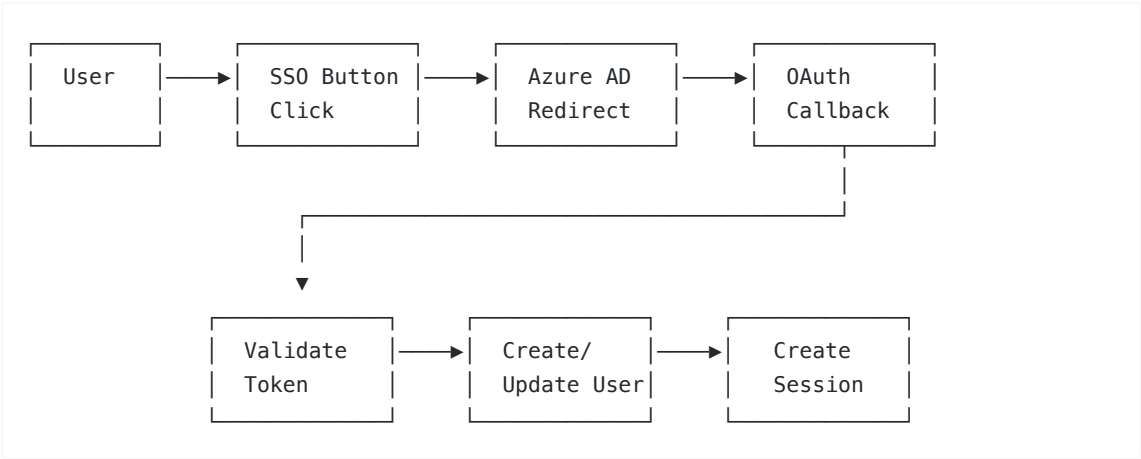
createMasterplan	masterplans.ts	Create new masterplan
updateMasterplan	masterplans.ts	Update existing masterplan
deleteMasterplan	masterplans.ts	Delete masterplan
getMasterplanEstimates	masterplan.ts	Calculate cost estimates
getCostModelEntries	costModel.ts	Fetch cost model data
getBenchmarkProjects	benchmarking.ts	Fetch benchmark projects
getConfiguration	configuration.ts	Get system configuration

3. Authentication Flow

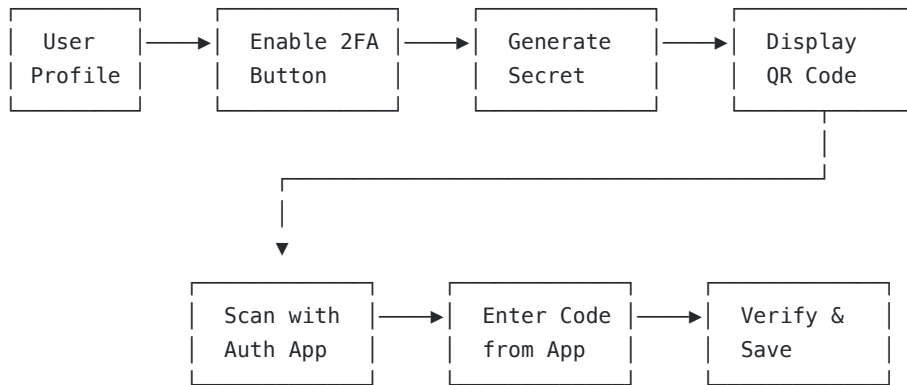
3.1 Standard Login Flow



3.2 SSO (Azure AD) Flow

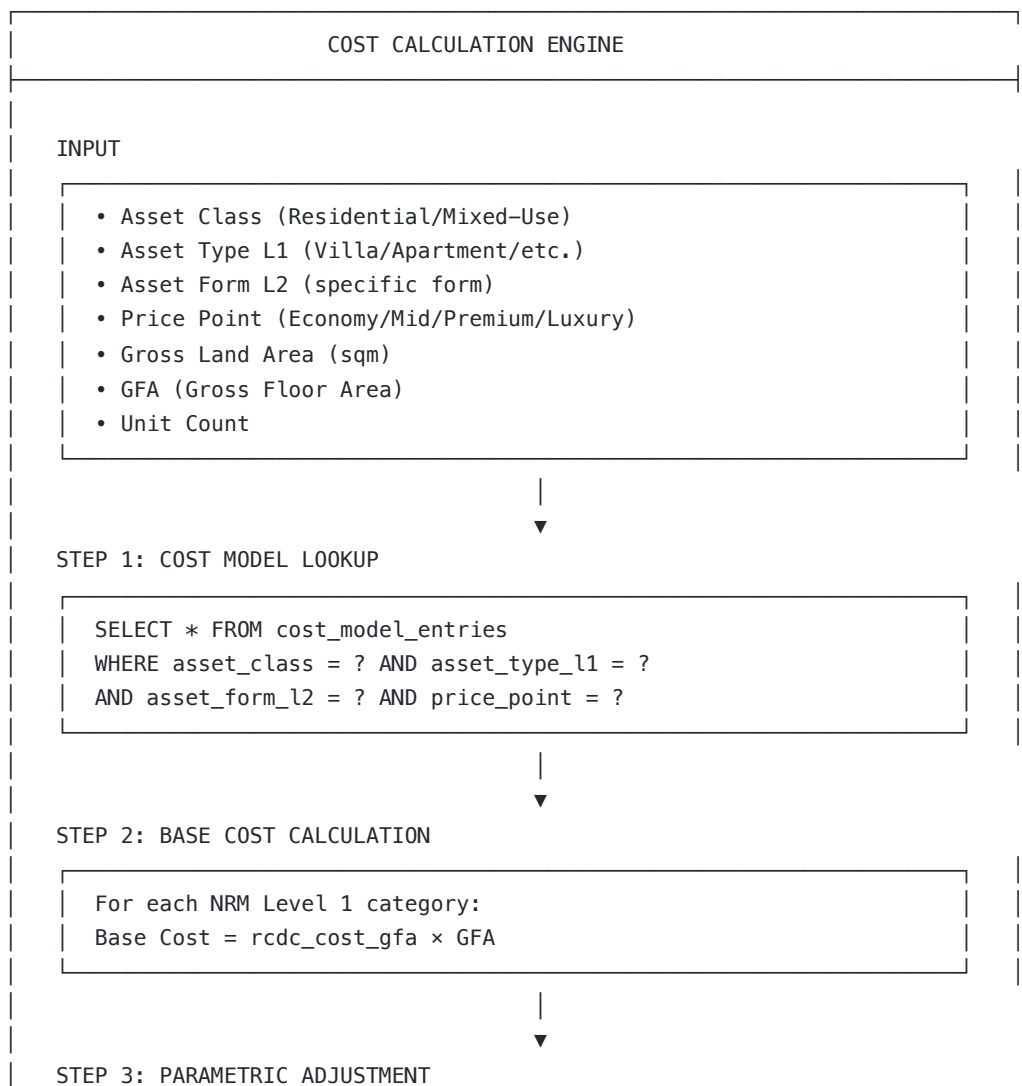


3.3 2FA Setup Flow



4. Calculation Engine

4.1 Cost Calculation Process



Adjusted Cost = Base Cost × Parametric Factor
(Factor from parametric_matrix table based on selections)



STEP 4: COST FACTOR UPLIFT

Final Cost = Adjusted Cost × Cost Uplift Factor
(Based on selected base date from cost_factors table)



OUTPUT

- Building Costs (by NRM category)
- Infrastructure Costs
- Car Parking Costs
- Public Realm Costs
- Other Costs (Contingency, Authority Fees, Soft Costs)
- Total Development Cost

4.2 NRM Categories

NRM Level 1	Description
0	Facilitating Works
1	Substructure
2	Superstructure
3	Internal Finishes
4	Fittings, Furnishings & Equipment
5	Services
6	Prefabricated Buildings
7	Work to Existing Building
8	External Works

4.3 Calculation Formulas

Building Asset Cost:

Building Cost = $\sum$ (Cost/GFA × Total GFA) for each NRM category

Infrastructure Cost:

$$\text{Infrastructure Cost} = \text{Base Rate} \times \text{GLA} \times \text{Density Factor}$$

Car Parking Cost:

Basement: $\text{Spaces} \times \text{Area per Space} \times \text{Cost per sqm}$
Podium: $\text{Spaces} \times \text{Area per Space} \times \text{Cost per sqm}$
Free-standing: $\text{Spaces} \times \text{Area per Space} \times \text{Cost per sqm}$

Public Realm Cost:

$$\text{Public Realm} = \text{Park Area} \times \text{Cost per sqm (based on park type)}$$

5. Component Architecture

5.1 Directory Structure

```
src/
├── app/                                # Next.js App Router
│   ├── (auth)/                        # Auth pages (login, etc.)
│   ├── (dashboard)/                  # Main application pages
│   │   ├── dashboard/                # Dashboard page
│   │   ├── masterplans/              # Masterplan pages
│   │   ├── projects/                 # Benchmark project pages
│   │   ├── configuration/            # Configuration pages
│   │   └── admin/                    # Admin pages
│   ├── api/                          # API routes
│   └── docs/                         # Documentation pages
├── components/                        # React components
│   ├── ui/                           # shadcn/ui components
│   ├── masterplan/                   # Masterplan components
│   ├── benchmarking/                 # Benchmark components
│   ├── configuration/                # Configuration components
│   ├── dashboard/                    # Dashboard components
│   └── admin/                        # Admin components
├── actions/                           # Server actions
│   ├── masterplan.ts                 # Masterplan calculations
│   ├── masterplans.ts                # Masterplan CRUD
│   ├── benchmarking.ts               # Benchmark operations
│   ├── costModel.ts                  # Cost model operations
│   └── configuration.ts               # Configuration operations
├── lib/                               # Utility libraries
│   ├── auth.ts                       # NextAuth configuration
│   ├── prisma.ts                     # Prisma client
│   └── utils.ts                       # Helper functions
```

```
└─ types/                # TypeScript types
   └─ index.ts            # Type definitions
```

5.2 Key Components

Component	Path	Description
MasterplanEstimator	components/masterplan/	Main cost estimation form
CostBreakdownTable	components/masterplan/	Display cost breakdown
PhaseTimeline	components/masterplan/	Phase planning interface
BenchmarkMap	components/benchmarking/	Map visualization
BenchmarkComparison	components/benchmarking/	Cost comparison charts
CostModelTable	components/configuration/	Cost model data table
UserManagement	components/admin/	User CRUD interface

6. Security Implementation

6.1 Password Security

```
// Password hashing with bcrypt
const SALT_ROUNDS = 12;
const hashedPassword = await bcrypt.hash(plainPassword, SALT_ROUNDS);

// Password validation
const isValid = await bcrypt.compare(plainPassword, hashedPassword);
```

6.2 Session Management

```
// NextAuth session configuration
export const authOptions: NextAuthConfig = {
  session: {
    strategy: "jwt",
    maxAge: 24 * 60 * 60, // 24 hours
  },
  jwt: {
    maxAge: 24 * 60 * 60,
  },
};
```

6.3 2FA Implementation

```
// TOTP secret generation
import { authenticator } from "otplib";
```

```
const secret = authenticator.generateSecret();
const otpauth = authenticator.keyuri(email, "ROSHN Platform", secret);

// Verification
const isValid = authenticator.verify({ token: userCode, secret });
```

6.4 Input Validation

```
// Using Zod for validation
const masterplanSchema = z.object({
  name: z.string().min(1).max(255),
  grossLandArea: z.number().positive(),
  totalUnits: z.number().int().positive(),
  assetClass: z.enum(["Residential", "Mixed-Use", "Commercial"]),
});
```

7. Error Handling

7.1 Error Types

Error Code	HTTP Status	Description
VALIDATION_ERROR	400	Invalid input data
UNAUTHORIZED	401	Not authenticated
FORBIDDEN	403	Not authorized
NOT_FOUND	404	Resource not found
CONFLICT	409	Duplicate resource
INTERNAL_ERROR	500	Server error

7.2 Error Handling Pattern

```
try {
  // Operation
  const result = await operation();
  return { success: true, data: result };
} catch (error) {
  if (error instanceof PrismaClientKnownRequestError) {
    // Handle database errors
    return handleDatabaseError(error);
  }
  if (error instanceof ZodError) {
    // Handle validation errors
    return { success: false, error: { code: "VALIDATION_ERROR", details:
error.errors } };
  }
}
```

```
    }  
    // Handle unknown errors  
    console.error("Unexpected error:", error);  
    return { success: false, error: { code: "INTERNAL_ERROR" } };  
  }  
}
```

8. Caching Strategy

8.1 React Query Configuration

```
const queryClient = new QueryClient({  
  defaultOptions: {  
    queries: {  
      staleTime: 5 * 60 * 1000, // 5 minutes  
      cacheTime: 30 * 60 * 1000, // 30 minutes  
      refetchOnWindowFocus: false,  
    },  
  },  
});
```

8.2 Cache Keys

Key Pattern	Data	TTL
["masterplans"]	Masterplan list	5 min
["masterplan", id]	Single masterplan	5 min
["benchmarks"]	Benchmark projects	10 min
["costModel"]	Cost model entries	30 min
["configuration"]	System configuration	30 min

9. Logging Specification

9.1 Audit Log Fields

Field	Type	Description
id	UUID	Unique log ID
userId	UUID	User who performed action
action	String	Action type (CREATE, UPDATE, DELETE, etc.)
entityType	String	Entity affected (masterplan, user, etc.)
entityId	UUID	ID of affected entity
oldValue	JSON	Previous state (for updates)

newValue	JSON	New state (for updates)
ipAddress	String	Client IP address
userAgent	String	Client user agent
createdAt	Timestamp	When action occurred

9.2 Logged Actions

Action	Entity Types
LOGIN	User
LOGOUT	User
CREATE	Masterplan, User, Benchmark
UPDATE	Masterplan, User, Configuration
DELETE	Masterplan, User, Benchmark
EXPORT	Masterplan, Report
IMPORT	Cost Model, Benchmark

10. Environment Configuration

10.1 Required Environment Variables

```
# Database
DATABASE_URL="postgresql://user:password@host:5432/database"

# Authentication
NEXTAUTH_SECRET="your-secret-key-min-32-chars"
NEXTAUTH_URL="https://platform.roshn.sa"

# Azure AD (Optional - for SSO)
AZURE_AD_CLIENT_ID="your-client-id"
AZURE_AD_CLIENT_SECRET="your-client-secret"
AZURE_AD_TENANT_ID="your-tenant-id"

# Mapbox (Optional - for maps)
NEXT_PUBLIC_MAPBOX_TOKEN="your-mapbox-token"

# Environment
NODE_ENV="production"
```

10.2 Configuration Table

System configuration stored in database:

Key	Type	Description
systemDefaults	JSON	Default calculation values
carParkingRates	JSON	Parking cost rates
infrastructureRates	JSON	Infrastructure cost rates
publicRealmRates	JSON	Public realm cost rates
otherCosts	JSON	Contingency, fees, soft costs

11. Deployment Checklist

11.1 Pre-Deployment

- ☐ Environment variables configured
- ☐ Database migrations applied
- ☐ Database seeded with initial data
- ☐ SSL certificates installed
- ☐ Firewall rules configured
- ☐ Backup strategy in place

11.2 Post-Deployment

- ☐ Health check endpoints responding
- ☐ Authentication working
- ☐ Database connectivity verified
- ☐ Logs being captured
- ☐ Monitoring configured
- ☐ Admin user created

11.3 Security Checklist

- ☐ HTTPS enforced
- ☐ Secure headers configured
- ☐ Rate limiting enabled
- ☐ CORS configured
- ☐ Input validation active
- ☐ Audit logging enabled